Before start, make sure you've read the following materials:

- **Tour of Go: Pointers and Structs (from "Pointers" until "Struct Literals")**
- **Tour of Go: Methods (from "Methods" until "Interfaces are implemented implicitly")**
- **Go FAQ: Should I define methods on values or pointers?**
- **Effective Go: Pointers vs. Values**
- **Effective Go: Embedding**
- **Go Wiki: Method Sets**
- **Functions and methods in Go**
- **Embedding in Go**
- **Composition and Forwarding in Go**
- **Go by Example: Structs, Methods, Struct Embedding**

Part 1: Structs

> **Important**
>
> ***Go has no classes. There is no `class`, no `extends`, no constructors. You group data with a `struct`, attach behaviour with methods, and build bigger types by composition.***

What is a struct?

A struct is a typed collection of named fields.

```
type Student struct {
    Name    string
    ID      int
    GPA     float64
    Courses []string
}
```

Creating struct values

```
var s1 Student                                    // zero value: every field is zero
s2 := Student{Name: "Aruzhan", ID: 1, GPA: 3.8}  // named fields (recommended)
s3 := Student{"Dias", 2, 3.2, []string{"Go", "Databases"}} // positional: ALL fields, in
order
p := &Student{Name: "Timur"}                      // pointer to a new struct
p.GPA = 3.5                                       // same as (*p).GPA = 3.5
q := new(Student)                                 // pointer to a zero Student
```

> **Tip**
>
> **Prefer named fields. A positional literal breaks as soon as someone adds or reorders a field.**

Key Points:

- **The zero value of a struct is ready to use: every field holds its own zero value.**
- **Go dereferences struct pointers for you: `p.GPA` works, you don't have to write `(*p).GPA`.**
- **Fields that start with a capital letter are exported (visible from other packages). Lowercase fields are private to the package.**

What will be the output here? What is the difference between `%v`, `%+v` and `%#v`?

```
fmt.Printf("%v\n", s1)
fmt.Printf("%+v\n", s2)
fmt.Printf("%#v\n", s3)
fmt.Println(s1.Courses == nil)
```

Structs are values

Assigning a struct, or passing it to a function, copies all of its fields. But remember Week 2: copying a slice field copies only the slice header — the underlying array is still shared.

> **Question**
>
> **What will be the output? Why did one field change in both structs, while the others didn't?**

```
type Team struct {
    Name    string
    Scores  [3]int
    Members []string
}

func main() {
    t1 := Team{Name: "Gophers", Scores: [3]int{1, 2, 3}, Members: []string{"Ali", "Bota"}}
    t2 := t1 // copy

    t2.Name = "Rustaceans"
    t2.Scores[0] = 100
    t2.Members[0] = "Nurlan"

    fmt.Println(t1.Name, t1.Scores, t1.Members)
    fmt.Println(t2.Name, t2.Scores, t2.Members)
}
```

Comparing structs

Structs can be compared with `==` only if all their fields are comparable (no slices, maps or functions).

```
type Point struct{ X, Y int }

a, b := Point{1, 2}, Point{1, 2}
c, d := &Point{1, 2}, &Point{1, 2}

fmt.Println(a == b)   // ?
fmt.Println(c == d)   // ?
fmt.Println(*c == *d) // ?
```

> **Question**
>
> **Predict the three results and explain them. Then try `t1 == t2` with the `Team` struct above. What does the compiler say?**

Anonymous and empty structs

```
point := struct{ X, Y int }{1, 2} // one-off struct without a type name

seen := map[string]struct{}{}     // a "set" (remember Week 3 maps)
seen["go"] = struct{}{}
_, ok := seen["go"]               // true
```

`struct{}` has no fields and takes 0 bytes, so `map[string]struct{}` is the idiomatic way to build a set in Go.

> **Task**
>
> ### Task 1: Student records
>
> 1. Declare the `Student` struct above and create students in 4 different ways (zero value, named fields, positional, pointer). Print each one with `%+v`.
> 2. Write `func addCourse(s Student, course string)` that appends a course to `s.Courses`. Call it and print the student. Did the course appear? Why?
> 3. Change the function to take `*Student` and call it again. What changed?

Part 2: Composition and embedding

> **Important**
>
> ***Go has no inheritance. Instead of "a Manager is an Employee", Go says "a Manager has an Employee". You build bigger types out of smaller ones.***

Composition with a named field

```
type Address struct {
    City string
}

type Person struct {
    Name    string
    Address Address // a normal, named field
}

p := Person{Name: "Ali", Address: Address{City: "Astana"}}
fmt.Println(p.Address.City) // Astana
```

Embedding: a field without a name

If you write only the type, without a field name, the type is embedded. Its fields and methods are promoted — you can use them directly on the outer struct.

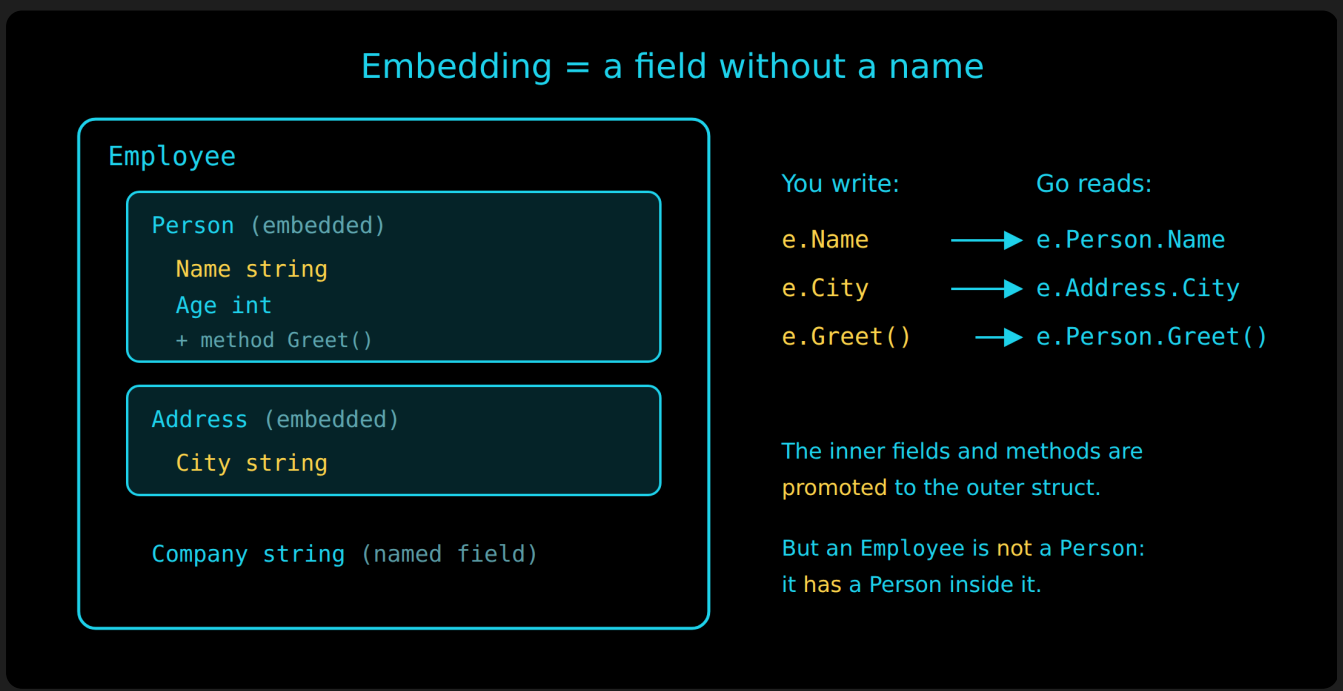
```
type Person struct {
    Name string
    Age  int
}

func (p Person) Greet() string { return "Hi, I am " + p.Name }

type Address struct {
    City string
}

type Employee struct {
    Person          // embedded: the field name is "Person"
    Address         // embedded: the field name is "Address"
    Company string  // named field
}

func main() {
    e := Employee{
        Person:  Person{Name: "Aigerim", Age: 21},
        Address: Address{City: "Almaty"},
        Company: "Gopher LLC",
    }
    fmt.Println(e.Name)        // Aigerim           — promoted field
    fmt.Println(e.Person.Name) // Aigerim           — full path still works
    fmt.Println(e.City)        // Almaty
    fmt.Println(e.Greet())     // Hi, I am Aigerim  — promoted method
}
```



> **Note**
>
> **An embedded field is still a real field, and its name is the type name (`e.Person`). Embedding only adds shortcuts; nothing is "merged".**

Shadowing

If the outer type declares a field or method with the same name, the outer one wins. The inner one is still reachable through the full path.

```
func (e Employee) Greet() string {
    return e.Person.Greet() + " and I work at " + e.Company
}

fmt.Println(e.Greet())        // Hi, I am Aigerim and I work at Gopher LLC
fmt.Println(e.Person.Greet()) // Hi, I am Aigerim
```

Name conflicts

If two embedded types at the same level have a field or method with the same name, and the outer type does not declare it itself, using that name is a compile error.

```
type Engine struct{}
func (Engine) Start() string { return "engine started" }

type Radio struct{}
func (Radio) Start() string { return "radio started" }

type Car struct {
    Engine
    Radio
}

c := Car{}
fmt.Println(c.Engine.Start()) // OK
fmt.Println(c.Start())        // compile error: ambiguous selector c.Start
```

Note: declaring `Car` is fine. The error appears only when you *use* the ambiguous name.

> **Caution**
>
> **Embedding is not inheritance!**

```go
type Animal struct{ Name string }

func (a Animal) Sound() string    { return "..." }
func (a Animal) Describe() string { return a.Name + " says " + a.Sound() }

type Dog struct{ Animal }

func (d Dog) Sound() string { return "Woof!" }

func main() {
    d := Dog{Animal{Name: "Rex"}}
    fmt.Println(d.Sound())
    fmt.Println(d.Animal.Sound())
    fmt.Println(d.Describe())
}
```

Question

What will be printed? Why doesn't the last line print `Rex says Woof!`?

Embedding a pointer

You can also embed a pointer type. Its methods are promoted in the same way — but if the pointer is `nil`, calling a promoted method that uses it panics.

```go
type Logger struct{ prefix string }

func (l *Logger) Log(msg string) { fmt.Println(l.prefix + msg) }

type Service struct {
    *Logger
    Name string
}

func main() {
    ok := Service{Logger: &Logger{prefix: "[svc] "}, Name: "users"}
    ok.Log("started") // [svc] started

    bad := Service{Name: "orders"} // Logger is nil!
    bad.Log("started")             // panic: nil pointer dereference
}
```

Task

Task 2: University

1. Create `Person` with fields `Name`, `Email` and a method `Contact() string` that returns `"Name <Email>"`.

2. Create `Student` that embeds `Person` and adds `GPA`. Create `Teacher` that embeds `Person` and adds `Department`.

3. Give `Teacher` its own `Contact()` that also includes the department. Call `Contact()` on a student, on a teacher, and call the "hidden" `Person.Contact()` of the teacher.

Participation Task: create a struct that embeds two types which both have a field `ID`. Show the compile error, then fix the code so that both IDs are printed.

Part 3: Methods and receivers

A method is a function with a receiver

The receiver is written between `func` and the method name. It is the value the method is called on.

```
type Rectangle struct{ Width, Height float64 }

func (r Rectangle) Area() float64 { return r.Width * r.Height }

// exactly the same logic as a plain function:
func Area(r Rectangle) float64 { return r.Width * r.Height }

func main() {
    r := Rectangle{3, 4}
    fmt.Println(r.Area())          // 12
    fmt.Println(Area(r))           // 12
    fmt.Println(Rectangle.Area(r)) // 12 — the receiver is just the first argument
}
```

> **Tip**
>
> **A method is syntactic sugar for a function whose first parameter is the receiver. Keep this in mind — it explains everything in the next part.**

Methods on non-struct types

You can declare methods on any named type defined in your package, not only on structs.

```
type Celsius float64

func (c Celsius) ToFahrenheit() float64 { return float64(c)*9/5 + 32 }

fmt.Println(Celsius(100).ToFahrenheit()) // 212
```

You cannot add methods to types from other packages, including built-in types:

```
func (i int) Double() int { return i * 2 }
// compile error: cannot define new methods on non-local type int
```

Receiver naming

- **Use a short name derived from the type: `r` for `Rectangle`, `c` for `Counter`.**
- **Use the same receiver name in all methods of one type.**
- **Don't use `this` or `self` — that's not idiomatic Go.**

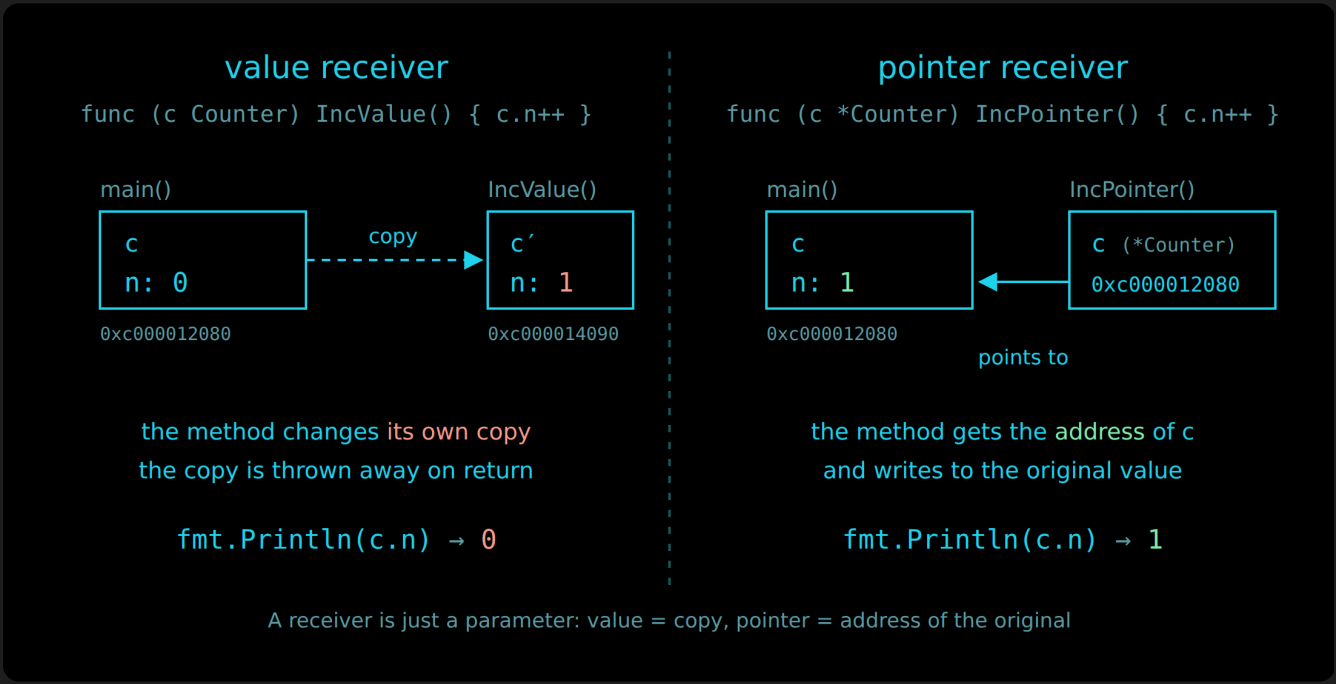
Part 4: Value vs pointer receivers

The receiver is a parameter, so the usual rule applies: a value is copied, a pointer is not.

```
type Counter struct{ n int }

func (c Counter) IncValue()    { c.n++ } // works on a COPY
func (c *Counter) IncPointer() { c.n++ } // works on the ORIGINAL

func main() {
    c := Counter{}
    c.IncValue()
    fmt.Println(c.n) // ?
    c.IncPointer()
    fmt.Println(c.n) // ?
}
```



Automatic `&` and `*`

Notice that we called `c.IncPointer()` on a value `c`, not on a pointer. Go helps you in both directions:

```
var c Counter
c.IncPointer()  // Go rewrites it as (&c).IncPointer()

p := &Counter{}
p.IncValue()    // Go rewrites it as (*p).IncValue()
```

Important

The automatic `&` only works if the value is addressable — it must live somewhere in memory that Go can point to.

Addressable: variables, slice elements `s[i]`, fields of addressable structs, `*p`.

Not addressable: map elements `m[k]`, function return values, struct literals like `Counter{}`.

Task

Task 3: Compile or not?

Without running the code, mark every numbered line as ✅ compiles or ❌ does not compile, and explain why. Then check yourself with `go build`.

```
type Counter struct{ n int }

func (c Counter) Get() int { return c.n }
func (c *Counter) Inc()    { c.n++ }

func NewCounter() Counter { return Counter{} }

func main() {
    var c Counter
    c.Inc()            // 1
    (&c).Inc()         // 2
    Counter{}.Get()    // 3
    Counter{}.Inc()    // 4
    NewCounter().Inc() // 5

    m := map[string]Counter{"a": {}}
    m["a"].Get()       // 6
    m["a"].Inc()       // 7

    s := []Counter{{}}
    s[0].Inc()         // 8

    pm := map[string]*Counter{"a": {}}
    pm["a"].Inc()      // 9
}
```

> ⚠ **Caution**
>
> **The `for range` copy trap**
>
> **`range` gives you a copy of each element. A pointer method called on that copy changes the copy, not the slice.**

```go
type Account struct {
    Owner   string
    Balance float64
}

func (a *Account) Deposit(amount float64) { a.Balance += amount }

func main() {
    accounts := []Account{{"Ali", 100}, {"Bota", 200}}

    for _, acc := range accounts {
        acc.Deposit(50)
    }
    fmt.Println(accounts) // ?

    for i := range accounts {
        accounts[i].Deposit(50)
    }
    fmt.Println(accounts) // ?
}
```

How to choose a receiver

Use a **pointer** receiver `(t *T)` when…	Use a **value** receiver `(t T)` when…
the method must **modify** the receiver	the type is small and behaves like a value (`Point`, `Celsius`)
the struct is **large** and copying it is expensive	the method only reads data and the type is cheap to copy
the struct contains something that must not be copied (`sync.Mutex`)	the type is a named basic, slice or map type (`type IDs []int`)
any other method already has a pointer receiver (consistency)	

Tip

Consistency rule from the Go FAQ: if some methods of a type need a pointer receiver, make all of them pointer receivers. Mixing both confuses readers and, as you'll see in Part 5, changes which interfaces the type satisfies.

Note

`go vet` catches one common mistake for you: a value receiver on a struct with a mutex. It reports `passes lock by value`.

Task

Task 4: Bank account

1. Create `BankAccount` with an exported `Owner` and an unexported `balance`.

2. Add `Deposit(amount float64)`, `Withdraw(amount float64) error` (return an error if there's not enough money) and `Balance() float64`.

3. Choose the receiver type for each method and justify your choice on the defence. Should `Balance()` have a value or a pointer receiver?

4. Put three accounts in a slice and add 5% interest to each one in a loop. Make sure the balances really change.

Task

Task 5: Benchmark value vs pointer receivers

In Week 3 you measured allocations. This time you'll measure copying.

```go
// Report is deliberately large: 1024 * 8 bytes = 8 KB.
type Report struct {
    Title  string
    Values [1024]float64
}

//go:noinline
func (r Report) FirstValue() float64 { return r.Values[0] }

//go:noinline
func (r *Report) FirstPointer() float64 { return r.Values[0] }
```

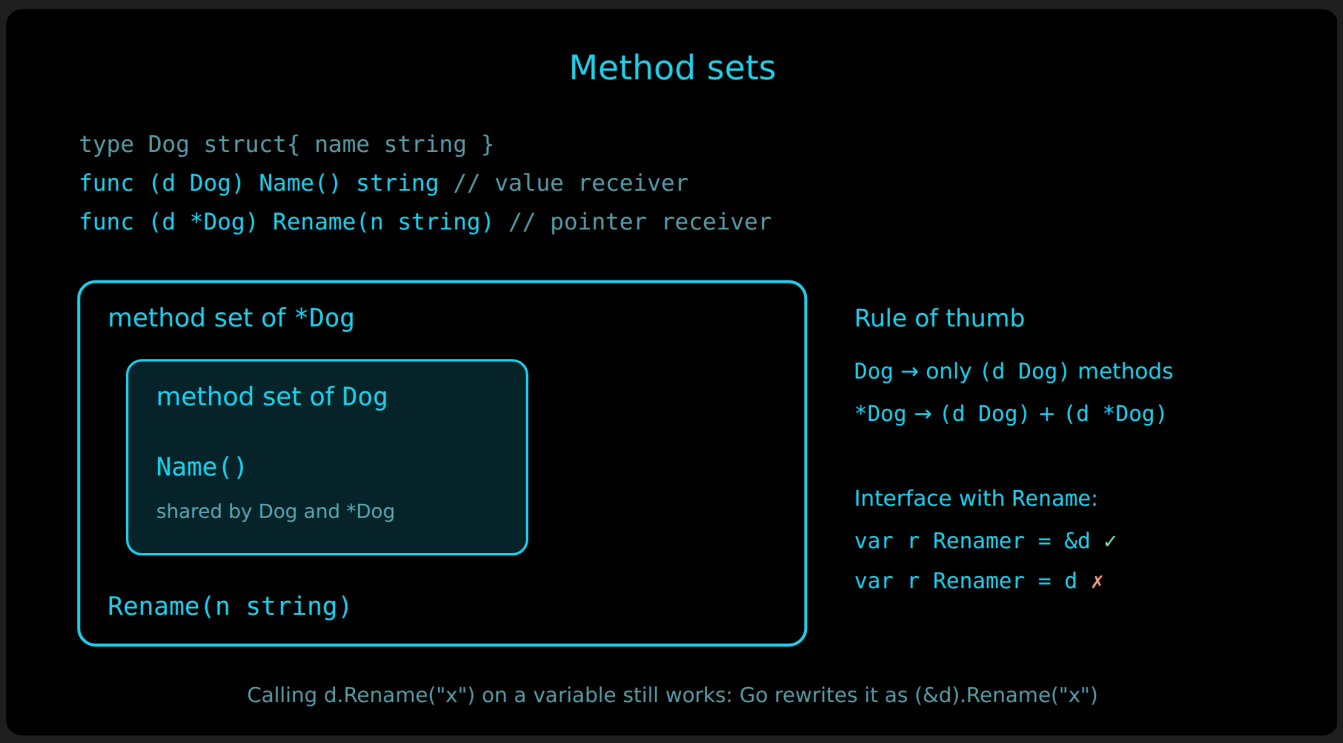
Requirements:

- **Write `BenchmarkFirstValue` and `BenchmarkFirstPointer` and run them with `go test -bench=. -benchmem`.**
- **Add the same two benchmarks for a small struct `Point{X, Y float64}`. Compare the four results.**
- **Explain: why is `B/op` 0 in all cases, even though the value receiver copies 8 KB each call? Where does the copy happen? Which column shows the cost?**
- **What does `//go:noinline` do, and why do we need it in this benchmark?**

Part 5: Method sets

The method set of a type is the list of methods you can call on a value of that type through an interface.

Type	Its method set contains
`T`	methods declared with receiver `T`
`*T`	methods declared with receiver `T` **and** `*T`



Why do we care? A first look at interfaces

An interface is a list of method signatures. A type implements an interface automatically if its method set contains all these methods — there is no `implements` keyword. (Interfaces are next week's topic; here we only need this much.)

```go
type Speaker interface {
    Speak() string
}

type Cat struct{ Name string }
func (c Cat) Speak() string { return c.Name + ": meow" }  // value receiver

type Dog struct{ Name string }
func (d *Dog) Speak() string { return d.Name + ": woof" } // pointer receiver

func main() {
    var s Speaker

    s = Cat{"Murka"}   // ✅ method set of Cat has Speak
    s = &Cat{"Barsik"} // ✅ method set of *Cat has Speak too
    s = &Dog{"Rex"}    // ✅ method set of *Dog has Speak
    s = Dog{"Sharik"}  // ❌ compile error
    fmt.Println(s.Speak())
}
```

```
cannot use Dog{…} (value of struct type Dog) as Speaker value in assignment:
Dog does not implement Speaker (method Speak has pointer receiver)
```

Question

We just saw that `d.Speak()` works on a `Dog` variable (Go adds `&` for us). So why can't a `Dog` value be put into a `Speaker`? Hint: the value inside an interface is a copy that is not addressable.

Tip

You can ask the compiler to check that a type implements an interface, without creating a real value:

```go
var _ Speaker = (*Dog)(nil) // fails to compile if *Dog stops implementing Speaker
```

Method sets and embedding

Struct `S` embeds…	method set of `S`	method set of `*S`
`T`	methods of `T`	methods of `T` and `*T`
`*T`	methods of `T` and `*T`	methods of `T` and `*T`

Question

Which of the lines A–D compile? Explain using the table.

```
type Speaker interface{ Speak() string }

type Base struct{}
func (Base) Hello() string  { return "hello" }
func (*Base) Speak() string { return "speak" }

type ByValue struct{ Base }    // embeds Base
type ByPointer struct{ *Base } // embeds *Base

var _ Speaker = &ByValue{}                // A
var _ Speaker = ByValue{}                 // B
var _ Speaker = ByPointer{Base: &Base{}}  // C
var _ Speaker = &ByPointer{Base: &Base{}} // D
```

Question

`fmt.Println` uses a type's `String() string` method if the type has one (the `fmt.Stringer` interface). What will this program print, and why are the two lines different?

```
type Book struct {
    Title  string
    Author string
}

func (b *Book) String() string { return b.Title + " by " + b.Author }

func main() {
    b := Book{"The Go Programming Language", "Donovan & Kernighan"}
    fmt.Println(b)
    fmt.Println(&b)
}
```

Task

Task 6: Shapes

1. Declare `type Shape interface { Area() float64; Perimeter() float64 }`.
2. Implement `Rectangle` with **value** receivers, plus a method `Scale(k float64)` that really changes the rectangle.
3. Implement `Circle` with **pointer** receivers.
4. Put one rectangle and one circle into a `[]Shape` and print the area and perimeter of each in a loop.
5. Try to put `Circle{Radius: 1}` (not `&Circle{...}`) into the slice. Read the compiler error and explain it.

Task

Task 7: Solve the problem from the tour of Go -> Exercise: Stringers

---

⭐ Bonus question (for those who want more)

```
r := Rectangle{3, 4}
f := r.Area   // a "method value"
r.Width = 10
fmt.Println(f(), r.Area()) // ?
```

What is printed? When exactly is the receiver copied?

Theoretical questions for the defence (in-person with Bakytzhan-agai):

- What is a struct? How is it different from a class in Java/C++?
- What happens when you assign one struct to another? What if the struct has a slice field?
- What does embedding do? Why is it **not** inheritance?
- What is the difference between a value receiver and a pointer receiver?
- Why can you call a pointer method on a variable, but not on a map element?
- What is the method set of `T`? Of `*T`?
- Why doesn't `Dog{}` implement an interface when `Speak` has a pointer receiver?
- When would you choose a value receiver?

Key takeaways:

- A struct is a value: assigning or passing it copies every field (but slice fields still share their array).
- Go uses composition, not inheritance. Embedding promotes fields and methods, but the outer type is not the inner type.
- A method is a function whose first parameter is the receiver.
- Value receiver = works on a copy. Pointer receiver = works on the original.
- Go adds `&` and `*` for method calls automatically, but only for addressable values.
- `range` gives you copies: use the index to modify slice elements.
- Method set of `T` = value-receiver methods. Method set of `*T` = all methods. This decides which interfaces a type implements.
- If one method needs a pointer receiver, make all of them pointer receivers.

Defence will be conducted in the next practice lesson, according to your schedule. In-person with your laptops.
